

MODULE 08 · DOMAIN 2 - IMPLEMENTATION AND INTEGRATION

The API Layer

Tasks 2.4 and 2.5.1. Streaming, async, throttling, backoff, fallback and model routing.

DOMAIN 2 of 5 EXAM WEIGHT 26% MODULE 8 of 22

A foundation model is **slow and rate-limited** compared with everything else in your architecture. Every pattern in this module exists because of that one asymmetry.

How a request can reach the model

SHAPE	API	USE IT WHEN
Synchronous	Converse, InvokeModel	A caller is waiting and the answer is short
Streamed	ConverseStream, InvokeModelWithResponseStream	A human is watching the answer appear
Asynchronous	SQS or Step Functions in front, or AsyncInvoke	Nobody waits, and the work must not be lost
Batch	Batch inference job, S3 in and S3 out	Bulk volume with no deadline. Half price

ON THE EXAM

Most integration questions are really asking which of these four rows fits. Work out who is waiting and how long they will tolerate waiting, and the row usually picks itself.

Streaming does not make the model faster. It changes **time to first token**, and time to first token is what a waiting human actually experiences.

Four ways to deliver a stream

OPTION	SHAPE	NOTE
Lambda Function URL	Direct HTTP, response streaming	Simplest. No Cognito user pool authoriser
API Gateway REST	Response streaming, since Nov 2025	Adds auth, throttling, usage plans
API Gateway WebSocket	Persistent bidirectional connection	Multi-turn sessions. Billed per connection-minute
AWS AppSync	GraphQL subscription with fan-out	Many subscribers per session. 30s resolver limit

ON THE EXAM

AppSync is the fan-out answer: one generated stream delivered to several subscribed clients. The others deliver to the one caller who asked.

The API Gateway streaming rule just changed

- ◆ **The old rule:** you cannot stream through API Gateway, so use a Function URL or a WebSocket API.
- ◆ **Since 19 November 2025,** API Gateway **REST** APIs support response streaming.
- ◆ That also brings integration timeouts up to **15 minutes** and payloads beyond the old 10 MB limit.
- ◆ **HTTP APIs still do not stream.** The distinction is REST versus HTTP, not API Gateway as a whole.
- ◆ Older question banks will still assume the old rule. If REST streaming is not offered, WebSocket is the intended answer.

ON THE EXAM

This is the third case of exam drift in the course, and it runs the same way as module 1's service tiers. Answer within the option set on the page.

A collaboration tool generates meeting summaries. When one participant requests a summary, every participant in the meeting should see it appear in real time. In the same place, as it is generated.

There may be up to forty participants in a meeting. The team already uses Amazon Cognito for authentication and wants to avoid managing connection state itself.

Which delivery mechanism fits?

- A A Lambda function URL with response streaming, called by each participant's client
- B AWS AppSync, with clients subscribing to a GraphQL subscription keyed by session
- C An API Gateway WebSocket API, storing each connection ID in DynamoDB and broadcasting to all of them
- D An API Gateway REST API with response streaming, polled by each client

◆ PAUSE AND DECIDE

Which delivery mechanism fits?

A Lambda function URL per client

Forty clients calling means forty generations of the same summary, at forty times the cost. It also lacks direct Cognito user pool auth.

C WebSocket API with connection IDs in DynamoDB

Achieves the same outcome by building the fan-out yourself, including connection storage and cleanup. The scenario rules that out in words.

B AppSync GraphQL subscription

Generate once, fan out to all subscribers. AppSync manages the connections, and it integrates with Cognito natively.

D REST streaming with polling

Polling a stream is a contradiction. Each poller either re-triggers generation or receives a partial response with no continuity.

THE SIGNAL

"Every participant should see it" is a fan-out requirement, and one generation feeding many subscribers is the only sensible reading. "Avoid managing connection state" then chooses AppSync over WebSocket.

When nobody is waiting

- ◆ **SQS in front of the model call** gives you buffering, retries, dead-letter queues and back-pressure.
- ◆ That buffer is what stops a traffic spike becoming a wall of `ThrottlingExceptions`.
- ◆ **Step Functions** when the async work has several steps, or needs a human callback.
- ◆ **EventBridge** to fan a single event out to several independent consumers.
- ◆ Return a **job ID immediately**, then deliver the result by notification, polling or subscription.

ON THE EXAM

A queue in front of a rate-limited dependency is the single most reliable pattern in this module. It converts "reject the request" into "process it slightly later".

Buffer in front, or fully asynchronous?

SYNCHRONOUS WITH A BUFFER

The caller still waits

- A concurrency limiter smooths the spike
- Response returns on the same connection
- Simpler client, no job IDs to track
- Under sustained load, waits become timeouts

FULLY ASYNCHRONOUS

The caller is released

- Job ID returned immediately
- Work survives a client disconnect
- Absorbs any spike the queue can hold
- Client must poll or subscribe

THE TIEBREAKER

A buffer helps a spike. It cannot help sustained overload, because a queue in front of a synchronous caller only converts throttling into waiting, and then into timeouts.

What Bedrock tells you when a call fails

ERROR	MEANING	CORRECT RESPONSE
<code>ThrottlingException, 429</code>	You exceeded RPM or TPM	Back off with jitter and retry
<code>ServiceQuotaExceededException, 400</code>	A quota boundary was crossed	Retrying will not help. Raise the quota
<code>ServiceUnavailableException, 503</code>	Temporary capacity constraint	Retry, and consider cross-Region inference
<code>ModelTimeoutException</code>	The model took too long	Retry, and reconsider prompt size
<code>ValidationException</code>	The request itself is wrong	Fix the request. Retrying repeats the error
<code>AccessDeniedException</code>	IAM or model access not granted	Fix the policy or request model access

ON THE EXAM

The first two look alike and behave oppositely. 429 is momentary and retryable. A 400 quota error is a boundary, and a retry loop against it just burns money.

Backoff, done properly

- ◆ **Exponential backoff with jitter.** Jitter is not optional. Without it, every client retries in the same instant.
- ◆ AWS SDKs support an **adaptive retry mode**, which is the built-in answer rather than hand-rolled loops.
- ◆ **Align retries to the 60-second window.** RPM refreshes per minute, so a retry three seconds later is usually pointless.
- ◆ **Track tokens, not just requests.** Long contexts hit TPM long before RPM.
- ◆ A retry loop with no ceiling turns a throttle into an outage, and bills you for every attempt.

ON THE EXAM

The jitter point is the one people skip. Synchronised retries from a hundred instances produce a thundering herd that guarantees the next round of throttling too.

Three applications in one AWS account share the same Bedrock model. Average combined traffic sits comfortably below the account's requests-per-minute quota, and a dashboard of daily averages looks healthy.

Every weekday between 09:00 and 09:20, all three applications peak together and users of the customer-facing application see failures. CloudWatch shows `ThrottlingException`. The team has already added exponential backoff with jitter, which reduced but did not eliminate the errors. The customer-facing application must never be the one that fails.

What should the team do?

- A Increase the retry count and lengthen the maximum backoff interval
- B Measure true peak RPM per application, request a quota increase sized to the summed peaks, and enforce per-application rate limits so the customer-facing app keeps its share
- C Move all three applications to Provisioned Throughput
- D Put an SQS queue in front of all three applications' model calls

What should the team do?

A More retries, longer backoff

Backoff smooths a transient spike. This is a sustained twenty-minute overload, so longer waits just move the failure later.

C Provisioned Throughput for all three

Buys guaranteed capacity for a twenty-minute daily peak and pays for it around the clock, including for two internal applications.

B Size the quota to summed peaks, plus per-app rate limits

Fixes the capacity shortfall and decides who loses when capacity is short. Without the second half, the noisiest app still starves the customer-facing one.

D SQS in front of all three

Correct for the batch applications and wrong for the customer-facing one, where a human is waiting and a queued response is a failed response.

THE SIGNAL

Averages hide coincident peaks. And "must never be the one that fails" is a prioritisation requirement, which no amount of extra capacity satisfies on its own.

What a resilient call actually does

01

Call

Invoke with a timeout that matches the model, not the default.

02

Retry

On 429 or 503, back off with jitter. Cap the attempts.

03

Fall back

Still failing? A smaller model, or another Region.

04

Trip

Repeated failures open the circuit. Fail fast, stop paying.

05

Degrade

Serve cached, templated or queued rather than an error.

ON THE EXAM

Each stage handles a longer failure than the one before it. Most production code implements stage two and stops, which is why a regional issue becomes a full outage.

Degrading gracefully

- ◆ **Model fallback.** Primary unavailable, drop to a smaller model rather than returning an error.
- ◆ **Cross-Region inference** absorbs a regional capacity shortfall without any code change.
- ◆ **Cached or template response** when the model is unreachable. A useful answer beats a spinner.
- ◆ **Circuit breaker.** After N consecutive failures, stop calling and fail fast until a probe succeeds.
- ◆ **AWS X-Ray** traces the call across service boundaries, which is how you find where the seconds went.

ON THE EXAM

Degradation must be designed, not discovered. Decide now what the application shows when Bedrock is unavailable, because the default is an unhandled exception reaching a customer.

Four ways to route to a model

APPROACH	DECIDED BY	REACH FOR IT WHEN
Static configuration	AppConfig or environment	One model per environment or tenant
Intelligent Prompt Routing	Bedrock, on predicted difficulty	Two models in one family, cost and quality
Step Functions	Explicit rules on content	Routing logic must be auditable and testable
API Gateway transformation	Request attributes at the edge	Routing by caller, plan or header

ON THE EXAM

Intelligent Prompt Routing is the managed one: GA since April 2025, you pick both models from one family and set the criteria, and AWS cites up to 30% cost reduction.

A SaaS platform serves a free tier and a paid tier from one API. Free-tier requests should use a small, cheap model with a low rate limit. Paid-tier requests should use a large model with a higher limit.

The tier is recorded in the caller's API key. The team wants the tier decision made before any model is invoked, wants per-tier rate limits enforced, and does not want tier logic duplicated across the several Lambda functions that call Bedrock.

How should the routing be implemented?

- ☐ A Use Amazon Bedrock Intelligent Prompt Routing to select the model based on predicted request complexity
- ☐ B Use API Gateway usage plans keyed to the API key for rate limiting, and route to the correct backend or pass the tier through, so no Lambda decides the tier itself
- ☐ C Have each Lambda function look up the caller's tier in DynamoDB and choose the model
- ☐ D Use AWS Step Functions to inspect each request and route by content complexity

◆ PAUSE AND DECIDE

How should the routing be implemented?

A Intelligent Prompt Routing

Routes on predicted difficulty of the prompt. The requirement routes on who is paying, which the content cannot tell you.

C Per-Lambda DynamoDB lookup

Duplicates tier logic across every function, which the scenario forbids, and adds a lookup to every request.

B API Gateway usage plans and edge routing

Usage plans tie rate limits to the API key directly, and the decision happens at the edge before any Lambda runs.

D Step Functions by content complexity

Again routes on content rather than entitlement, and inserts an orchestration layer into a single synchronous call.

THE SIGNAL

Routing by content and routing by caller are different problems. Two of these options solve the first when the scenario describes the second, which is the whole discrimination.

The awkward details of an FM-facing API

- ◆ **Token limits** are a design constraint. Reject or truncate oversized input at the edge, not at the model.
- ◆ **Model timeouts** are longer than typical HTTP defaults. Client, gateway and function timeouts must all agree.
- ◆ **API Gateway usage plans** give per-caller throttling and quotas, which protects your Bedrock quota from one client.
- ◆ **Request validation** at the gateway rejects malformed input before it costs a model invocation.
- ◆ A 29-second gateway timeout in front of a 40-second generation fails every time, and the model still bills you.

ON THE EXAM

Timeout laddering is a classic production bug. The client gives up at 30 seconds, the gateway at 29, the function at 60, and the model completes at 40 having been paid for and discarded.

A document analysis API accepts a document and returns a structured assessment. Behind an API Gateway REST API, a Lambda function calls Bedrock synchronously. The Lambda timeout is 120 seconds.

Short documents work. Long documents consistently return a 504 to the client after about 29 seconds, and CloudWatch shows the Lambda function continuing to run and completing successfully at around 55 seconds. Bedrock is billed for every one of these invocations. Users may wait, but must receive the result.

What is the correct fix?

- ☐ A Reduce the Lambda timeout to 29 seconds so it fails at the same time as the gateway
- ☐ B Make the endpoint asynchronous: return a job ID immediately, process via SQS, and deliver the result by polling or notification
- ☐ C Request an API Gateway integration timeout increase from AWS Support
- ☐ D Switch to a faster model so long documents complete within 29 seconds

◆ PAUSE AND DECIDE

What is the correct fix?

A Lower the Lambda timeout to 29 seconds
Makes the failure tidier and still a failure. The result is never delivered, which is the one thing the scenario says must happen.

C Request a timeout increase
Response streaming does raise the ceiling, but this endpoint returns one structured object at the end, so there is nothing to stream progressively.

B Asynchronous with job ID and SQS
"Users may wait, but must receive the result" is the definition of async. The work completes and the answer is delivered, instead of being computed and discarded.

D Faster model
Buys headroom against a fixed 29-second wall that longer documents will breach again. It treats a design mismatch as a performance problem.

THE SIGNAL

The 29-second figure and the successful 55-second completion together are the diagnosis. Work that outlives its caller's timeout is asynchronous work, whatever the endpoint currently claims.

Client configuration nobody reads until it breaks

- ◆ **Connection pool size.** An exhausted pool surfaces as 503s that look like a Bedrock capacity problem.
- ◆ **Client-side read timeout** must exceed the model's generation time, or you abandon calls you paid for.
- ◆ **Reuse the client object.** Constructing one per invocation adds handshake latency to every request.
- ◆ **Adaptive retry mode** and a raised max-attempts value are one configuration block, not custom code.
- ◆ Defaults are tuned for fast APIs. Every one of them is wrong for a call that takes forty seconds.

ON THE EXAM

The connection pool point is the non-obvious one, and AWS calls it out directly: pool exhaustion produces 503s that send teams hunting for a capacity problem that does not exist.

The GenAI gateway

- ◆ **One internal service** in front of Bedrock, which every application calls instead of calling Bedrock directly.
- ◆ It centralises **rate limiting per team**, retries, fallback, model routing and prompt version selection.
- ◆ It gives you **cost attribution**, because every call carries a known caller identity.
- ◆ It is where **guardrails and logging** are applied once rather than in every application.
- ◆ The cost is a hop, a team to own it, and a single point of failure to design around.

ON THE EXAM

Skill 2.3.5 names GenAI gateway architectures explicitly. Scenario 2's per-application rate limiting is the same idea arrived at from the throttling direction.

An enterprise has eleven teams calling Bedrock directly with their own IAM roles. Nobody can say which team is responsible for which portion of the bill, and guardrail configuration differs between teams. Two teams have also hardcoded a model that reaches end of life in four months.

A platform team has been asked to fix all three problems. They have limited headcount and want to avoid a change that requires all eleven teams to rewrite their applications simultaneously.

What should the platform team build?

- A An internal GenAI gateway service, adopted team by team, applying guardrails, model selection and per-team attribution centrally
- B A service control policy denying `bedrock:InvokeModel` except for approved model IDs
- C Cost allocation tags on each team's IAM role, plus documentation of the required guardrail configuration
- D Move all eleven teams into separate AWS accounts so each has its own bill and quota

What should the platform team build?

- A

 GenAI gateway, adopted incrementally
One place that solves all three, and teams migrate one at a time because the gateway is an endpoint change rather than a rewrite.
- B

 SCP restricting model IDs
Solves the model lifecycle problem by breaking two teams' applications on the day it is applied. No attribution, no guardrail consistency.
- C

 Tags plus documentation
Tags give attribution. Documentation is not enforcement, so guardrail drift and hardcoded models both continue.
- D

 Eleven separate accounts
Gives per-team bills and quotas, and multiplies guardrail configuration by eleven. A large migration that solves one of three problems.

THE SIGNAL

Three problems and a headcount constraint. The gateway is the only option that addresses all three and can be adopted incrementally, which is what "not simultaneously" demands.

The five things to carry forward

- ◆ **Who is waiting, and for how long?** That question picks sync, streamed, async or batch.
- ◆ **Streaming fixes perceived latency.** Function URL, REST streaming, WebSocket, or AppSync for fan-out.
- ◆ **429 means back off. A 400 quota error means raise the quota.** They look alike and behave oppositely.
- ◆ **Jitter, adaptive retry, and the 60-second window.** Track TPM as well as RPM.
- ◆ **Route by content or route by caller** — decide which, because different services answer each.

ON THE EXAM

Module 9 covers deployment: Bedrock throughput modes against SageMaker endpoint types, container patterns for LLMs, and model cascading.